

ReturnProMax GCP Deployment Plan

Prepared for ReturnProMax - Shopify and Fynd returns app | Date: May 27, 2026

The recommended GCP architecture is a Cloud Run service backed by Cloud SQL for PostgreSQL, with Secret Manager, Cloud Scheduler, Cloud Build, Artifact Registry, and optional Memorystore Redis. GKE is not recommended for the current app shape because it adds operational overhead without solving a real scaling or runtime problem.

Recommendation: Start with Cloud Run plus Cloud SQL. Split database migrations and startup backfills out of the web container before production traffic. The current app can run on Cloud Run, but repeated startup backfills on autoscaled instances are not a clean production pattern.

Application Runtime

Area	Current Repo Behavior
Application type	Single Node.js React Router service serving Shopify admin UI, customer portal APIs, Fynd webhooks, and cron endpoints.
Node version	<code>node >=22.12</code>
Build command	<code>npm install && npx prisma generate && npm run build</code>
Current start command	<code>npm run start</code> , which runs Prisma migrations, startup backfills, and then <code>react-router-serve</code> .
Database	PostgreSQL via Prisma, configured through <code>DATABASE_URL</code> .
Optional cache/rate limit store	Redis via <code>REDIS_URL</code> . Without Redis, rate limiting is per Cloud Run instance.

Target GCP Services

Cloud Run

Runs the public web service for Shopify, Fynd webhooks, and portal routes. Use min instances 1 for lower cold-start risk.

Cloud SQL for PostgreSQL

Primary relational database for shops, sessions, return cases, webhook logs, settings, and encrypted credentials.

Secret Manager

Stores production secrets: Shopify keys, JWT secret, encryption key, database URL, cron secret, and webhook secrets.

Artifact Registry

Stores versioned container images built from the repository.

Cloud Build

Cloud Scheduler

Runs CI/CD: install, test, typecheck, build image, push image, deploy service.

Calls internal cron endpoints with `CRON_SECRET` on a fixed schedule.

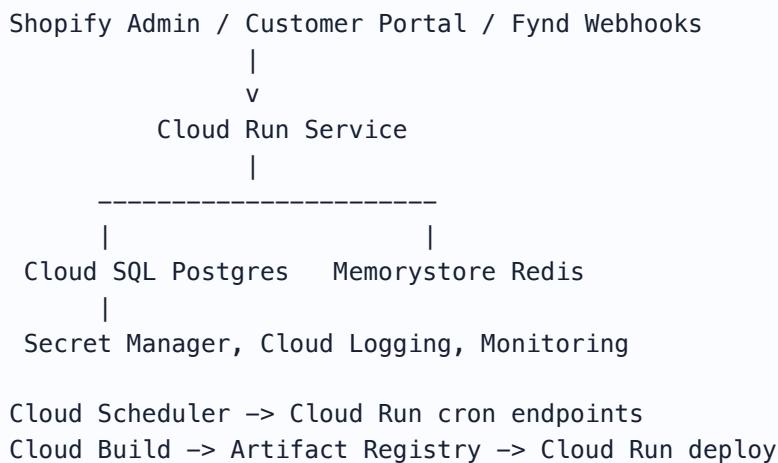
Memorystore Redis

Optional but recommended for strict cross-instance rate limiting.

Cloud Logging and Monitoring

Captures app logs, deploy failures, 5xx alerts, webhook failures, refund failures, and database saturation signals.

High-Level Architecture



Required Environment Variables

Variable	Purpose
<code>NODE_ENV=production</code>	Ensures production runtime behavior.
<code>DATABASE_URL</code>	Cloud SQL PostgreSQL connection string. Use SSL/private connectivity where possible.
<code>SHOPIFY_API_KEY</code>	Shopify app client ID/API key from Partner Dashboard.
<code>SHOPIFY_API_SECRET</code>	Shopify app secret for OAuth and webhook verification.
<code>SCOPES</code>	Comma-separated Shopify scopes required by the app.
<code>SHOPIFY_APP_URL</code>	Canonical public app URL, for example <code>https://returns.example.com</code> .
<code>PORTAL_JWT_SECRET</code>	Random 32-byte hex value for portal JWT signing.
<code>ENCRYPTION_KEY</code>	Random 32-byte hex value for encrypting stored Fynd and notification credentials.

<code>CRON_SECRET</code>	Bearer/header secret used by Cloud Scheduler when invoking cron endpoints.
<code>PORTAL_CSRF_REQUIRED=true</code>	Keep portal mutations protected in production.

Generate cryptographic secrets with `openssl rand -hex 32`. Do not rotate `ENCRYPTION_KEY` without a credential rotation plan, because existing encrypted merchant credentials depend on it.

Optional Environment Variables

Variable	When to Use
<code>REDIS_URL</code>	Use with Memorystore Redis for cluster-wide rate limiting.
<code>FYND_WEBHOOK_SECRET</code>	Legacy global Fynd webhook secret. Per-shop DB secrets should remain preferred.
<code>LOG_LEVEL=info</code>	Controls Pino logging verbosity.
<code>BUILD_VERSION</code> , <code>BUILD_COMMIT</code>	Useful for release traceability in logs.
<code>OTEL_SERVICE_NAME</code> , <code>OTEL_ENVIRONMENT</code>	Use when exporting traces and metrics to an observability backend.

Cloud Scheduler Jobs

Endpoint	Suggested Schedule	Authentication
<code>POST /api/fynd-webhook-retry-cron</code>	Every 15 minutes	Authorization: Bearer <code><CRON_SECRET></code>
<code>POST /api/fynd-consolidation-cron</code>	Hourly	Authorization: Bearer <code><CRON_SECRET></code>
<code>GET /api/scheduled-report</code>	Daily or hourly	<code>x-cron-secret: <CRON_SECRET></code>

Deployment Pipeline

1. Create Artifact Registry repository for the app image.
2. Create Cloud SQL PostgreSQL instance and database.
3. Create Secret Manager entries for all required secrets.
4. Create Cloud Run service with Node 22 container image and Cloud SQL connectivity.
5. Configure Shopify Partner Dashboard URLs using the Cloud Run or custom domain URL.
6. Create Cloud Scheduler jobs for retry, consolidation, and scheduled reports.
7. Add log-based alerts for webhook errors, refund errors, 5xx spikes, and DB connection saturation.

Production Correction Before Launch

The current `npm run start` runs `prisma migrate deploy` and startup backfills before starting the server. On Cloud Run, new instances can start during scaling or redeployments, which can repeat startup backfill attempts. The production setup should split this into a Cloud Run Job or Cloud Build deploy step:

Migration/backfill job: `prisma migrate deploy` and controlled backfills.

Web service: start only `react-router-serve ./build/server/index.js`.

Minimum Viable GCP Setup

Start with Cloud Run, Cloud SQL PostgreSQL, Secret Manager, Artifact Registry, Cloud Build, and Cloud Scheduler. Add Memorystore Redis once traffic grows or strict rate limiting matters across multiple Cloud Run instances.

Launch Checklist

- Cloud Run service responds at `/`.
- Shopify OAuth install and callback succeed.
- Customer portal return creation works against production URL.
- Fynd per-shop webhook URL is reachable and signature verification is configured.
- Cloud Scheduler jobs return successful JSON responses.
- Prisma migrations are applied before web traffic is shifted.
- `npm audit --omit=dev`, lint, typecheck, and build pass in CI.
- Alerts exist for app 5xx, webhook retry exhaustion, refund failures, and Cloud SQL CPU/connection pressure.